

Announcing Helidon Talon

OCI's next generation framework for building OCI Native Services

What Is Helidon Talon?

- Helidon extensions for building OCI Native Services
- A versioned BOM for consistent adoption across service teams
- Less infrastructure wiring in every application

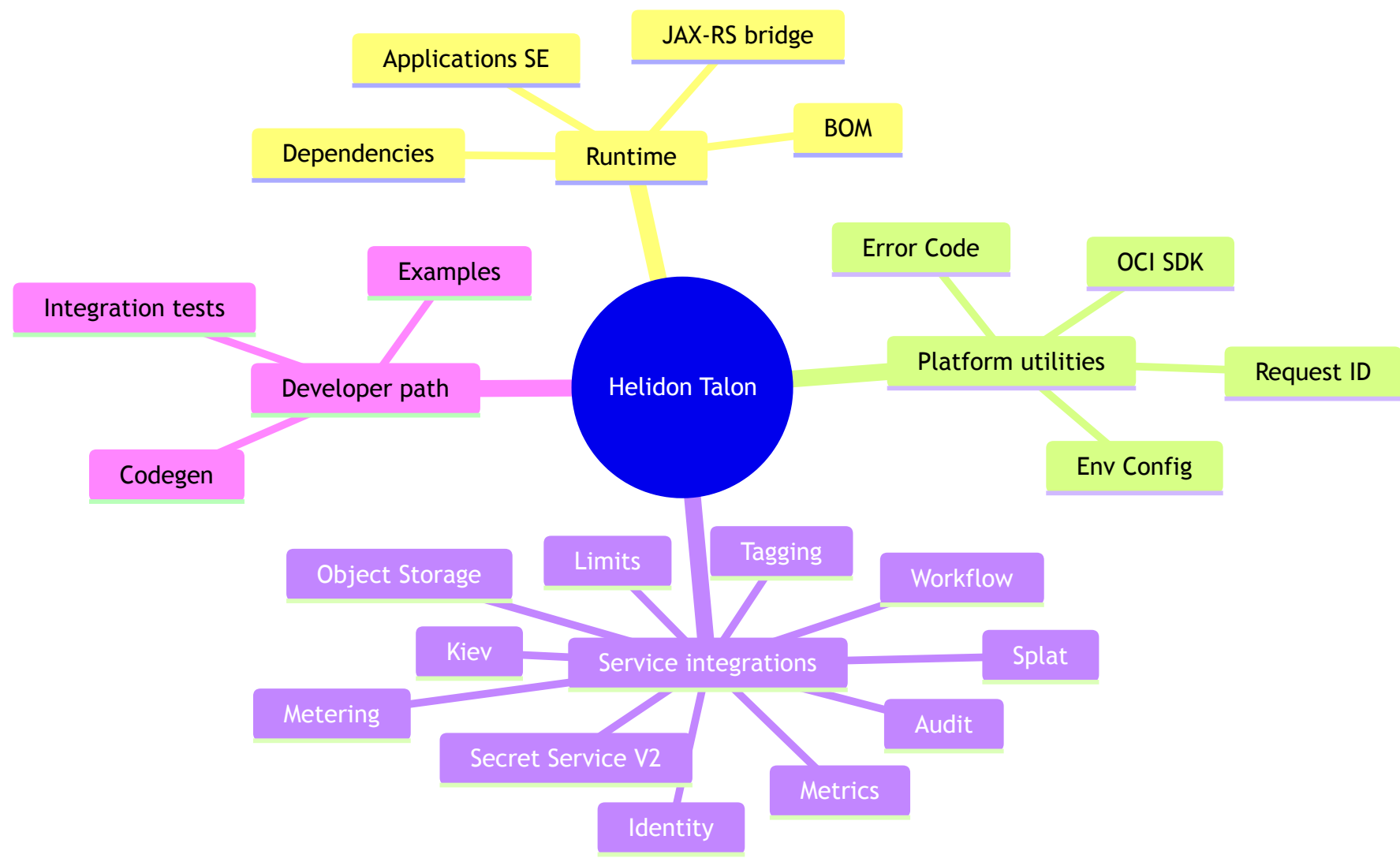
Provides

- OCI integrations and injectable clients
- Environment config, auth, and observability

Targets

- Helidon services running in OCI
- DP services first; stronger CP support next

At a glance



Helidon 4 Foundation

Programming model

- Helidon SE-first services
- Compile-time injection via the service registry
- Typed configuration with generated metadata
- Direct service code with platform interceptors

Runtime model

- JDK 25 baseline
- High-performance request handling with virtual threads
- Blocking code instead of async/reactive plumbing
- Helidon infrastructure for routing, lifecycle, config, and observability

Baseline and Size

Helidon 4

Target runtime, currently `4.5.0-M1`

28+

Managed artifacts in the BOM

JDK 25

Compiler and documented baseline

13+

Documented service integrations

OCI SDK 2.88.0

Managed through dependencies

9+

Runnable example applications

OCI Environment Config

- Discovers and publishes OCI location data
- Keeps region/domain handling out of application code
- Supports runtime discovery, dynamic core-regions metadata, IMDS fallback, and test overrides

Value class	Examples
Region	Public OCI region, internal region name, SDK <code>Region</code> service
Location	Availability domain and fault domain
Domains	Public domain and IaaS domain for endpoint construction
Overrides	Local/test <code>location-override</code> values in <code>oci-config.yaml</code>

How `oci-env` Is Used

Discover

Region, AD, FD, public domain, and IaaS domain

Publish

Config keys such as
`oci.env.region-name`

Consume

Clients derive endpoints and SDK region services

- Keeps region and domain logic out of client integration code
- Supports runtime discovery, IMDS fallback, dynamic metadata, and test overrides
- Example: Secret Service can use `${oci.env.iaas-domain-name}`

OCI SDK Authentication

- Shared authentication selection for OCI Java SDK clients
- Exposes the selected `BasicAuthenticationDetailsProvider`
- Auth methods are contributed by classpath modules
- Client integrations consume one provider contract

OCI SDK Authentication Configuration

- Config root is `helidon.oci`
- `authentication-method` selects or requires a provider
- `auto` tries available methods from the service registry
- `BasicAuthenticationDetailsProvider` can be injected

```
helidon:  
  oci:  
    authentication-method: auto  
    allowed-authentication-methods:  
      - instance-principal  
      - resource-principal  
    region: us-ashburn-1
```

Service Integrations

Filters + Interceptors

**Injectable Clients +
Config**

**Declarative API +
Codegen**

Helidon Service Registry

Audit

- Inserts a Helidon WebServer filter for OCI Audit v2 events
- Captures request path, action, request id, caller, response status, and timing
- Uses config rules to whitelist audited parameters and headers
- Emits events through the Sherlock audit logging path
- Respects `oci-splat-audited` to avoid duplicate SPLAT audit work
- Lets endpoint logic enrich the current audit event
 - `AuditPayloadAppender` is available in request scope

Audit Configuration

- `oci.auditv2` enables auditing and names the audit source
- Rules explicitly whitelist audited request values
- A rule applies only when resource and action match

```
oci:
  auditv2:
    enabled: true
    event-source: MyService
    request-parameter-rules:
      - resources: "/orders"
        actions: "POST"
        values: "orderId"
```

Identity

- Integrates Helidon endpoints with the OCI Auth SDK
- Provides authentication, authorization, and SPLAT-aware request handling
- Registers `Principal`, `AuthorizationRequest`, and `IdentityContext` per request
- Uses annotations and built-time generated interceptors

Identity Configuration

- Config root is `oci.identity`
- Authentication and authorization are configured independently
- Region and location defaults can come from `oci-env`

```
oci:
  identity:
    authentication:
      global-business-unit: "Cloud-Infra"
      team-name: "ExampleTeam"
      application-name: "MyService"
      use-instance-principal: true
    authorization:
      enabled: true
      service-name: "my-service"
```

Identity Example

- Auth SDK annotations drive generated authorization interceptors
- Request data can be injected directly into resource methods

```
@RestServer.Endpoint
@Path("/orders")
class OrdersEndpoint {

    @Http.POST
    @AuthorizationPermission("ORDER_CREATE")
    String create(@Http.Entity CreateOrder request,
                  Principal principal) {
        return principal.getSubjectId();
    }
}
```

Kiev

- Provides Helidon configuration and service registry bindings for Kiev
- Supports in-memory, direct database, and Kiev-as-a-service backends
- Exposes `DataStore`, `MappedDataStore`, and transaction support
- Adds `@KievTransaction` for declarative transaction handling

Kiev Configuration

- Config root is `oci.kiev`
- Each entry defines one named data store
- Store names qualify injected Kiev services and transactions

```
oci:
  kiev:
    data-stores:
      - backend: "IN_MEMORY"
        store-name: orders-store
        app-name: MyService
```

Kiev Example

- Inject `MappedDataStore` by configured store name
- Open mapped buckets from service code

```
@Service.Singleton
class OrderStore {

    @Service.Inject
    OrderStore(@Service.Named("orders-store")
               MappedDataStore dataStore) {
        var bucket = dataStore.getOrCreateBucket("orders",
                                                  String.class,
                                                  Order.class);
    }
}
```

Kiev Transactions

- `@KievTransaction` opens the transaction boundary around a service method
- The generated interceptor supplies a `Transaction` parameter when one is declared
- Successful write transactions commit on method exit
- Failures abort in-flight writes, then close the transaction

```
@KievTransaction(value = "orders-store", name = "order-put")
Order put(Transaction tx, Order order) {
    return bucket.put(tx, order);
}
```

Limits

- Provides Helidon integration for OCI Limits
- Exposes a configured Limits DP client
- Uses shared OCI SDK authentication and client configuration
- Supports endpoint and timeout configuration

Limits Configuration

- Config root is `oci.limits`
- Client settings use the common OCI SDK timeout shape
- Endpoint can be overridden for explicit routing

```
oci:
  limits:
    endpoint: https://limits.example.oraclecloud.com
    client:
      connection-timeout: PT5S
      read-timeout: PT30S
```

Limits Example

- Inject the configured `LimitsDPClient`
- Use it from service code to check limits and quotas

```
@Service.Singleton
class LimitsService {
    private final LimitsDPClient limitsClient;

    @Service.Inject
    LimitsService(LimitsDPClient limitsClient) {
        this.limitsClient = limitsClient;
    }

    List<ServiceLimit> serviceLimits() {
        GetServiceLimitsRequest request = GetServiceLimitsRequest.builder()
            .group("compute")
            .tag("vm-standard")
            .build();
        return limitsClient.getServiceLimits(request).getItems();
    }
}
```

Metering

- Reports OCI service usage through native metering libraries
- DP is high volume and must avoid per-request metering calls
 - Accumulates usage efficiently and reports emitter-dp batches
 - Uses Object Storage for durable backup
- CP is lower traffic and matches emitter-cp start/stop style
 - Supports annotations for points, timers, and metered regions
 - Uses Kiev through a `MappedDataStore` supplier

Metering Configuration (DP)

- Config root is `oci.metering`
- Required values come from the native emitter-dp builder
- Object Storage backup can be configured separately when enabled

```
oci:
  metering:
    enabled: true
    endpoint: "https://bling.example.internal"
    metering-dir: "/var/opt/oracle/metering"
    client-id: "orders-dp"
    service: "orders"
    os-enabled: false
    k8s-based-deployment: false
```


Metering Example (DP)

- DP services inject the native `LogFileUsageRecorder`
- Service code records native emitter-dp meter batches

```
@Service.Singleton
class OrderMetering {

    private final LogFileUsageRecorder recorder;

    @Service.Inject
    OrderMetering(LogFileUsageRecorder recorder) {
        this.recorder = recorder;
    }

    void recordBatch(List<Meters> meters) throws Exception {
        recorder.recordMetersAsync(meters).call();
    }
}
```

Metering Configuration (CP)

- Uses metering-agent with Kiev-backed log stores
- Metered calls run inside a Kiev transaction

```
oci:
  metering:
    enabled: true
    endpoint: "https://bling.example.internal"
    client-id: "orders-cp"
    region: "us-phoenix-1"
    metering-period: "PT1M"
    bucket-configs:
      - bucket-name: "orders_metering"
        service-name: "orders"
        meter-name: "orders.requests"
```

Metering Example (CP)

- CP metering annotations are processed at compile time
- Parameters identify compartment, resource, amount, and tags

```
@Metering.Point(value = "orders.created",  
                tags = @Metering.Tag(key = "source", value = "api"))  
void createOrder(@Metering.CompartmentId String compartmentId,  
                @Metering.ResourceId String orderId,  
                @Metering.Amount float amount,  
                @Metering.TagValue("operation") String operation) {  
    // create order  
}
```

Metrics

- Publishes Helidon neutral metrics to OCI metrics
- Works with normal Helidon metrics APIs and annotations
- Supports counters, timers, distribution summaries, gauges, and functional counters
- Adds automatic HTTP request metrics and optional JVM gauges

Metrics Configuration

- Configured as a Helidon metrics publisher of type `oci`
- Project and fleet identify the emitted metric stream
- Region can be explicit or supplied through `oci-env`
- Endpoint can be derived from OCI Monitoring or explicitly overridden

```
metrics:  
  publishers:  
    - type: oci  
      project: my-service  
      fleet: my-fleet  
      region: us-ashburn-1
```

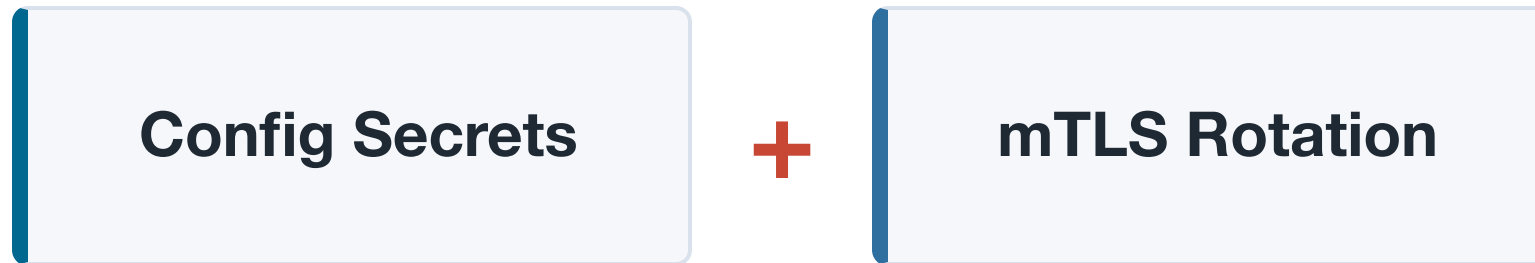
Metrics Example

- Service code uses Helidon metrics annotations
- Helidon Talon publishes metric updates to OCI metrics

```
@Http.GET
@Path("/hello/{name}")
@Metrics.Timed(value = "personalized-greeting",
               absoluteName = true)
String personalizedGreeting(@Http.PathParam("name") String name) {
    return "Hello, " + name + "!";
}
```

Secret Service V2

- Exposes Secret Service V2 values through Helidon Config
- Provides server and client mTLS material



Secret Service V2 Secrets

- Exposes Secret Service V2 values through Helidon Config
- Maps a configured key prefix to SSv2 secret paths
- Resolves secrets lazily when application code reads the key
- Caches direct reads and polls only previously requested keys
- Can derive endpoints from environment configuration

Secret Service V2 Secrets Configuration

- Configured under `helidon.oci-secret-service`
- Prefix maps config lookups to Secret Service paths
- Endpoint can use `${oci.env.iaas-domain-name}`

```
helidon:
  oci-secret-service:
    prefix: "oci.ssv2"
    cache-ttl: "PT5M"
    client:
      endpoint: "https://secret-service-ce.${oci.env.iaas-domain-name}/v1"
```

Secret Service V2 Secrets Example

- Inject Helidon Config
- Read SSv2-backed values using the configured prefix

```
@Service.Singleton
final class SecretAwareService {
    private final String password;

    @Service.Inject
    SecretAwareService(Config config) {
        this.password = config.get("oci.ssv2/secret/orders/db/latest")
            .asString()
            .orElseThrow();
    }
}
```

Secret Service V2 mTLS

- Provides the `oci-ssv2` TLS manager
- Loads PKI JSON material from SSv2 or a Helidon resource
- Rotates server and client mTLS material on a reload schedule
- Supports Helidon clients and raw `SSLContext` use

Secret Service V2 mTLS Rotation

- Select `oci-ssv2` under Helidon TLS manager config
- `pki.secret-path` points to PKI JSON material in SSv2
- `reload.expression` controls the refresh cadence
- The same manager shape works for server and client TLS

```
server:
  tls:
    client-auth: "REQUIRED"
    manager:
      oci-ssv2:
        reload.expression: "0 0/30 * * * ? *"
        pki.secret-path: "/secret/orders/mtls/server/latest"
        trust.path: "/etc/oci-pki/ca-bundle.pem"
```

Splat

- Adds SPLAT mTLS validation support for Helidon endpoints
- Bridges SPLAT-authenticated requests into Helidon request handling
- Supports OCI service front-door integration patterns
- Works with Identity for SPLAT-aware authentication flows

Splat Configuration

- Config root is `oci.splat`
- Validation runs on generated endpoint handlers
- Region can be explicit or resolved from the OCI environment

```
oci:  
  splat:  
    enabled: true  
    reject-x-region-calls: false  
    region: us-ashburn-1
```

Tagging

- Contributes the internal tagging client to the Helidon service registry
- Converts tag sets to and from persisted tag slugs
- Supports freeform, defined, and system tags
- Helps connect resource write paths with authorization flows

Tagging Configuration

- Config root is `oci.tagging`
- Client is available from the Helidon service registry
- Metrics emission can be enabled or disabled

```
oci:  
  tagging:  
    emit-metrics: false
```


Tagging Example

- Inject `TaggingClient` from the service registry
- Convert request tags into a persisted tag slug

```
@Service.Singleton
class TagService {
    private final TaggingClient taggingClient;

    @Service.Inject
    TagService(TaggingClient taggingClient) {
        this.taggingClient = taggingClient;
    }

    byte[] slug(Map<String, String> tags) {
        TagSet tagSet = TagSet.builder()
            .freeformTags(FreeformTags.builder()
                .tags(tags)
                .build())
            .build();
        return taggingClient.toByteArray(tagSet);
    }
}
```

Workflow

- Integrates OCI Workflow-as-a-Service with Helidon Talon services
- Inject the native WFaaS `WorkflowClient` directly
- Use worker and poller clients through Helidon service registry names
- Configure endpoint, domain, timeouts, retry policy, and worker identity

Workflow Configuration

- Config root is `oci.workflow`
- Endpoint settings configure WFaaS routing and timeouts
- Worker and poller clients are exposed through service registry names

```
oci:
  workflow:
    domain-id: compute-control-plane
    endpoint-details:
      server-endpoint: https://wfaas.example.internal
      connect-timeout: PT5S
    worker-identifier: my-service-worker
```

Workflow Example

- Inject default worker `WorkflowClient`
- Serialize JSON request with Helidon `JsonBinding`

```
@Service.Singleton
class ProvisioningWorkflowService {
    private static final JsonBinding JSON = JsonBinding.create();
    private final WorkflowClient workflowClient;

    @Service.Inject
    ProvisioningWorkflowService(WorkflowClient workflowClient) {
        this.workflowClient = workflowClient;
    }

    WorkflowInstance launch(LaunchWorkflowRequest json) {
        byte[] payload = JSON.serialize(json).getBytes(StandardCharsets.UTF_8);
        LaunchWorkflowArguments request = LaunchWorkflowArguments.builder()
            .workflowDefinitionId(WorkflowDefinitionId.builder()
                .name("instance-create")
                .build())
            .workflowArguments(payload)
            .build();
        return workflowClient.launchWorkflow(request);
    }
}
```

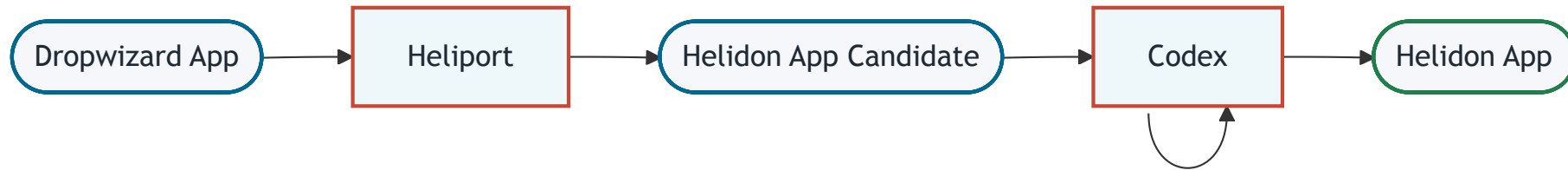
Integrated Client Versions

Client library	Managed version
OCI Java SDK	2.88.0
Identity Auth SDK	3.1.336
Kiev	2.0.35
Limits DP Java Client	1.9
Metrics module	2.0.12
Metrics library	1.1.30

Client library	Managed version
Workflow / WFaaS	14.0.0
Metering DP emitter	1.0.0.9
Metering CP agent	11.0.0.68
SPLAT SDK	0.1.39635
Tagging	4.3.5
Vault user Java client	1.0.8

Helipoint

- Migrates Dropwizard apps to Helidon Talon apps
- Combines OpenRewrite recipes with Codex-guided planning and validation
- Produces a migrated tree, `.helipoint/` report, action plan, and proof evidence
- Still improving and can benefit from a Codex analysis after migration



Data Plane Reference App

- Runnable Helidon Talon reference service under `examples/data-plane`
- Combines request ID, OCI error responses, Identity, Kiev, and Audit
- Demonstrates signed robot create/update/delete APIs with `@AuthorizationPermission`
- Uses in-memory Kiev by default so teams can run and test without external infrastructure

What Teams Should Take Away

- Consider Heliport + Codex for Dropwizard-to-Helidon migration work
- Consult the Data Plane Reference App before wiring a new DP service
- Let `helidon-oci-envconfig` resolve region, AD, FD, and domains
- Add only the Helidon Talon integrations your service needs
- Stay tuned for additional features and integrations in upcoming releases!

Resources

- User's Channel: `#helidon-users`
- Documentation: `https://helidon.oraclecorp.com/docs/2.0.0-RC1/`
- Repository: `https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/oci-helidon`
- Heliport: `https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/heliport`